

UNIVERSITY OF KARACHI
Department of Computer Science
Software Project Management — 506 (LAB 3+0)

CONGINE

Deterministic Output Governance for AI-Assisted Software Development

PROJECT CONCEPT BRIEF

An Overview of the Problem Domain, Product Concept and Solution Approach

Field	Detail
Document Reference	CGN-DOC-01
Project Name	Congine — Deterministic Output Governance Engine
Course	Software Project Management — 506 (LAB 3+0)
Instructor	Dr. Humera
Semester	6th
Assignment	Assignment-01: Project Charter Development
Team Members	1. _____ Roll No: _____ 2. _____ Roll No: _____ 3. _____ Roll No: _____ 4. _____ Roll No: _____ 5. _____ Roll No: _____
Version	1.0
Date of Submission	_____
Status	Submitted for Evaluation

Table of Contents

1. Executive Summary
2. Introduction and Background
3. Problem Statement
4. The Congine Concept
5. Problem Domain I — Token Optimization
6. Problem Domain II — Tracking History and Project Memory
7. Problem Domain III — Multi-Agent Normalization
8. How the Three Domains Relate
9. Conceptual Working of the System
10. Intended Users and Usage Scenarios
11. Distinguishing Characteristics
12. Project Scope and Current Status
13. Conclusion
14. Glossary of Terms

1. Executive Summary

Congine is a deterministic output governance engine for software teams that use artificial intelligence coding assistants. It is designed to sit between a development team and the AI systems that team relies on, and to enforce — automatically, consistently and verifiably — that the output produced by those AI systems conforms to the organization's own standards before it is accepted into a codebase or a downstream system.

The motivation for the project arises from a structural change in how software is now written. AI coding assistants have moved rapidly from experimental novelty to everyday production tooling. They generate code, structured data, configuration and documentation at a speed no human team can match. What they do not do is guarantee that what they produce respects the specific architectural rules, data contracts, naming conventions, or compliance requirements of the organization consuming that output. The result is a governance gap: velocity has increased dramatically while assurance has not.

Congine addresses that gap through a simple and deliberately strict principle. Organizational requirements are expressed as machine-readable contracts written and controlled by human engineers. Every relevant output is checked against the applicable contract by a rule-based validation engine before it is allowed to proceed. The verdict is deterministic: for a given output and a given contract, the outcome is always identical. It does not vary between runs, and it is not produced by another probabilistic model.

The Central Proposition

A compliance or engineering lead cannot operate on the statement “our AI outputs are probably correct.” They require the statement “our outputs conform to our rules, or they are stopped.” Congine is built to make the second statement true. Determinism is therefore not merely a technical preference within this project; it is the defining property from which the rest of the design follows.

Beyond the core enforcement function, the project addresses three connected problem areas that emerged from studying how development teams actually work with AI assistants day to day: the economic cost of repeated and wasteful interaction with AI systems, the loss of project context and decision history across working sessions and between team members, and the inconsistency that arises when different developers within the same organization use different AI assistants. These three areas are introduced individually in Sections 5 to 7 and their interdependence is explained in Section 8.

This document provides a conceptual overview suitable for readers approaching the project for the first time. It is accompanied by two further documents: a technical specification describing the system's architecture and requirements, and a business and project management document containing the formal project charter, business case, risk assessment and delivery plan.

2. Introduction and Background

2.1 The Shift in How Software Is Produced

For most of the history of commercial software development, the unit of production was a human engineer writing code, and the unit of assurance was a human engineer reviewing it. Quality control processes — code review, static analysis, testing, continuous integration — were designed around the assumptions that code is produced at human speed, in human quantities, and with human error patterns.

That set of assumptions no longer holds. AI coding assistants now generate substantial volumes of code within professional engineering teams. A single developer working with such an assistant can produce, in an afternoon, a quantity of code that would previously have represented a week of work. The productivity gain is real and measurable, and it is the reason adoption has been so rapid.

However, the assurance mechanisms surrounding that production have not scaled at the same rate. Human review remains the primary control, and human review is inherently slow, inconsistent and subject to fatigue. When the volume of generated output rises by an order of magnitude while the review capacity remains constant, the proportion of output that receives genuine scrutiny falls. This is the structural condition that Congine is designed to address.

2.2 The Nature of the Assurance Problem

It is important to characterize the problem precisely, because an imprecise characterization leads to the wrong solution. The difficulty is not that AI assistants produce code that fails to run. Modern assistants are, on the whole, competent at producing syntactically valid and functionally plausible output. The difficulty is that functional correctness and organizational correctness are different properties.

An AI assistant has no knowledge of the internal conventions of the organization it is working for. It does not know that a particular module is forbidden from importing another, that a particular response field must conform to a specific enumerated set of values, that a particular category of output must carry a regulatory disclaimer, or that a particular naming convention is mandatory. Unless that knowledge is supplied explicitly and enforced mechanically, the assistant will produce output that runs correctly and violates the organization's rules simultaneously — and such violations are precisely the kind that pass unnoticed in a rushed review.

Accumulated across many developers and many interactions, these individually minor deviations produce architectural drift: a codebase that gradually diverges from its intended design without any single decision having caused the divergence.

2.3 Why an Automated Enforcement Layer

Three properties are required of any mechanism intended to close this gap, and they jointly explain the shape of the proposed system.

- **It must be automatic.** A control that depends on a human remembering to apply it will be applied inconsistently, and inconsistency is the condition the control exists to eliminate.
- **It must be fast.** A control that materially slows the development loop will be disabled by the developers it is intended to protect. Any check must complete quickly enough to be invisible in normal use.
- **It must be deterministic.** A control whose verdict varies between identical inputs cannot be the basis of an organizational policy, cannot be meaningfully audited, and cannot be trusted by the teams subject to it.

The third property is the most consequential and the most frequently compromised. A natural instinct when confronted with the problem of governing AI output is to apply a second AI system as the reviewer. This approach is rejected in the present design, because it inherits the same probabilistic uncertainty that created the problem,

introduces latency into the development loop, and produces verdicts that cannot be defended in an audit. The validation performed by Congine is therefore rule-based and executed in ordinary program code.

3. Problem Statement

Problem Statement

Software organizations that have adopted AI coding assistants lack any automatic, fast and deterministic mechanism to ensure that the output produced by those assistants conforms to the organization's own architectural, structural and policy requirements. Consequently, non-conforming output enters production systems undetected; developers expend avoidable time and computational cost correcting output through repeated trial and error; project knowledge and design rationale are lost between working sessions and between team members; and teams using different assistants produce work of divergent quality and style within a single codebase.

3.1 Decomposition of the Problem

The problem statement above decomposes into four distinct sub-problems. The first is the core enforcement problem; the remaining three are the connected problem domains that define the wider scope of the project.

#	Sub-Problem	Observable Symptom	Consequence if Unaddressed
P0	Absence of automated enforcement	Output violating internal standards reaches the codebase without detection	Progressive architectural drift; latent defects; failed audits
P1	Inefficient interaction cost	Developers repeat instructions to the assistant several times before an acceptable result is obtained	Wasted developer time and avoidable operational expenditure
P2	Loss of project context	Each working session begins by reconstructing what was previously done and why	Repeated re-derivation of solved problems; slow onboarding; lost design rationale
P3	Inconsistency across assistants	Different developers using different assistants produce work of differing style and quality	A codebase with multiple internal dialects; uneven quality; unpredictable review burden

Sub-problem P0 is the foundation of the system and is addressed by the core validation engine. Sub-problems P1, P2 and P3 correspond to the three problem domains described in Sections 5, 6 and 7 respectively. They are not independent features but interdependent capabilities that build upon the enforcement foundation, as explained in Section 8.

4. The Congine Concept

4.1 Definition

Congine in One Sentence

Congine intercepts an output produced by an AI system, evaluates it against a contract written by a human engineer, and determines whether that output is permitted to proceed — rapidly, deterministically, and without disrupting the application in which it operates.

4.2 The Governing Analogy

The system is most readily understood by analogy with a network firewall. A firewall is positioned between an untrusted external network and a protected internal network. When a packet arrives, the firewall evaluates it against a configured ruleset. Conforming packets are forwarded; non-conforming packets are rejected. The protected network never receives traffic that the ruleset does not permit.

Congine applies this structure to the output of AI systems. The packet is the output produced by the assistant. The ruleset is the contract authored by the engineering team. The firewall is the validation engine. The protected network is the codebase or downstream system that consumes the output.

The property of a firewall that matters most in this analogy is determinism. For a given packet and a given ruleset, a firewall always produces the same decision. It is this reliability, rather than sophistication, that makes a firewall a viable basis for organizational security policy. Congine is designed to offer the same reliability for AI output.

4.3 The Contract

The contract is the central artifact of the system and the mechanism by which control is returned to human engineers. A contract is a machine-readable document, authored and version-controlled by the engineering team, that specifies the requirements an output must satisfy: which fields must be present, what types and value ranges they may take, which values are permitted for constrained fields, and which structural patterns must be respected.

Three properties of the contract are worth emphasizing. It is written by humans, which means the standard being enforced is a deliberate organizational decision rather than an inference made by a model. It is machine-readable, which means it can be evaluated automatically at scale. And it is the single point of reference against which every output is judged, irrespective of which AI system produced that output — a property that becomes significant in Section 7.

Contracts and Human Authority

A recurring concern regarding AI-assisted development is the gradual transfer of engineering authority from people to systems. The contract mechanism is the project's direct answer to that concern. The organization's standards are stated explicitly, in a document that engineers own, review and revise. The AI system is not consulted about what the standard should be; it is measured against it.

4.4 The Verdict

When an output is evaluated against a contract, the engine produces a structured result describing whether the output conformed and, if it did not, precisely which requirements were violated. The consuming system then applies a configured enforcement policy to that result. Three broad enforcement postures are supported in

principle: permissive operation, in which violations are recorded and reported but the output is allowed to proceed; strict operation, in which a violating output is blocked outright; and observational operation, in which validation runs and is recorded but never interferes with execution.

The availability of graduated enforcement is a deliberate adoption strategy. An organization introducing the system into an existing codebase can begin in an observational posture, accumulate evidence about the frequency and nature of violations, and progress to strict enforcement once the contracts have been proven accurate. A tool that can only operate in its most restrictive mode is unlikely to be adopted at all.

5. Problem Domain I — Token Optimization

5.1 Statement of the Problem

Interaction with an AI language model carries a direct and measurable cost. That cost is levied on the quantity of text exchanged, measured in units called tokens, and it is charged for both the text sent to the model and the text the model returns. For an individual developer the cost of a single interaction is negligible. For an organization operating at scale, the aggregate is a significant and rapidly growing operational expense.

5.2 Underlying Cause

The principal driver of this expense is a structural characteristic of how these systems operate: an interaction with a language model does not retain memory of previous interactions. Each request is processed independently, and any context the model requires must be supplied again with every request.

An analogy clarifies the consequence. Consider a highly capable consultant who possesses complete expertise but retains no memory between meetings. Every time the consultant is engaged, the entire case file must be presented again from the beginning. The consultant reads it, provides an excellent response, and retains nothing. The re-presentation of the file is not optional, and it is charged for on every occasion.

This effect is amplified considerably by iteration. When an AI assistant produces an unsatisfactory result, the developer typically restates the request with additional guidance and the assistant attempts the task again. Each attempt requires the entire accumulated context to be transmitted and processed once more. A task requiring several attempts therefore incurs several times the cost of a task completed correctly on the first attempt, and the largest component of that expenditure is the repeated retransmission of material that has not changed.

Where the Expenditure Actually Occurs

The dominant cost in AI-assisted development is not the successful interaction but the unsuccessful one — the corrective cycle in which the assistant does not know that its output is unacceptable and therefore continues to produce variations of it. Reducing the number of such cycles is consequently more valuable than reducing the size of any individual request.

5.3 Solution Approach

The project addresses this cost through several complementary mechanisms operating on the same underlying principle: the number and size of unproductive interactions should be reduced.

- **Early and definitive detection.** Because the validation engine evaluates outputs using program logic rather than an AI model, a non-conforming output can be identified immediately and at no token cost whatsoever. The assistant is informed that its output is unacceptable without a further model interaction being required to establish that fact.
- **Precise corrective information.** A verdict that merely reports failure obliges the assistant to search for the fault. A verdict that identifies exactly which requirement was violated, and in which part of the output, permits a targeted correction. Convergence in a single corrective attempt rather than several is the single largest source of saving available.
- **Economical context construction.** Where an interaction is genuinely required, only the relevant portion of the material is transmitted rather than the entire body of surrounding context, substantially reducing the size of each request.

- **Cooperation with provider-side caching.** Modern AI providers offer mechanisms that avoid reprocessing portions of a request that have not changed since a previous request. Structuring interactions so that stable material is positioned consistently allows these mechanisms to be used effectively.

An observation worth recording is that the enforcement capability described in Section 4 and the cost-reduction capability described here are not separate features requiring separate implementation. The deterministic validation engine, built to solve the assurance problem, is simultaneously the principal mechanism for reducing expenditure, because eliminating a wasted corrective cycle eliminates its entire cost. The economic benefit is largely a consequence of the correctness mechanism rather than an addition to it.

6. Problem Domain II — Tracking History and Project Memory

6.1 Statement of the Problem

Software projects are worked on by multiple people across extended periods. Each working session, and each transfer of work between individuals, requires the reconstruction of context: what has been completed, what remains, which approaches were previously attempted and rejected, and why the system is structured as it is.

In an organizational setting this reconstruction is a recurring and substantial cost. A developer resuming work after an interval must re-establish the state of the project. A developer inheriting work from a colleague must infer decisions that were never recorded. A new team member must reconstruct months of accumulated reasoning before becoming productive. The cost is paid repeatedly and is rarely measured.

6.2 Underlying Cause

Two distinct causes converge on the same symptom, and distinguishing them is necessary to design a coherent solution.

- **Human context loss.** Decisions and their justifications are typically held in the memories of individuals and in transient communication channels rather than in durable, searchable records. When an individual is unavailable, or when a communication channel is cleared, the knowledge is irrecoverable.
- **Machine context loss.** The AI assistant, as described in Section 5, retains nothing between sessions. It has no knowledge of the project's architecture, previous decisions, or the approaches already found unsuitable, unless that information is supplied again on each occasion.

The second cause is the same statelessness that drives the cost problem of Section 5. This is a significant observation: two of the project's three problem domains originate in a single underlying property of the technology.

6.3 Solution Approach

The project treats this as a requirement for durable institutional memory — a record that survives the unavailability of any individual and the ending of any session.

Conceptually the design comprises three complementary records. The first is a permanent, chronologically ordered log of validation events, recording what was evaluated, against which contract, with what outcome, and in what circumstances. This log is written once and never altered, which is what makes it dependable as an audit record. The second is a representation of the project's current structure — its components and the permitted relationships between them — enabling a proposed change to be assessed against the architecture as it presently exists rather than as it was assumed to be. The third is an analytical view derived from the first, identifying recurring patterns of violation and areas of the codebase that are repeatedly problematic.

The design must satisfy two constraints simultaneously. The record must be complete enough to be genuinely useful, and the portion of it supplied to an AI assistant must be small enough to remain economical. The resolution is selective recall: rather than supplying the entire history, the system supplies only the portion relevant to the immediate task, in a compact form. This connects the domain directly to Section 5, since the material recalled is precisely the material whose transmission cost must be controlled.

Privacy by Design

A permanent record of development activity is a sensitive artifact. The design principle adopted is that the historical record should retain the minimum information necessary to be useful. Wherever the substance of an output is not required for later analysis, a cryptographic digest is retained in place of the content

itself. This reduces both the privacy exposure and the consequences of any future compromise of the record.

7. Problem Domain III — Multi-Agent Normalization

7.1 Statement of the Problem

Within a single organization, different developers commonly use different AI assistants. The choice is influenced by individual preference, licensing arrangements, prior familiarity and organizational policy, and it is unlikely to converge on a single tool.

These assistants are not equivalent. Presented with the same task, they produce different results: different structural choices, different naming conventions, different handling of edge cases, and different levels of adherence to instructions. The consequence for the organization is a codebase that gradually acquires several internal dialects, and a review burden that varies unpredictably according to which assistant produced the work under review.

7.2 The Question of Approach

The intuitive response to this problem is to propose that the governance system should learn the behavioural characteristics of each assistant and adapt to them individually. This project deliberately rejects that approach at the core of the system, for reasons that are worth stating explicitly.

- Such an approach would require a substantial volume of labelled behavioural data that does not exist at the outset of the project.
- It would represent a research undertaking of uncertain duration rather than an engineering deliverable of predictable scope.
- Most importantly, it would introduce probabilistic behaviour into precisely the component whose value depends upon being deterministic, thereby undermining the property established in Section 4 as foundational.

7.3 Solution Approach

The project instead achieves normalization through mechanisms that are deterministic in nature. The reasoning is best expressed by analogy.

Consider an organization that recruits engineers trained in different countries, under different educational traditions. The organization does not attempt to retrain each recruit from first principles. It applies three simpler measures. It requires every recruit to satisfy the same certification standard. It provides, where necessary, translation so that each individual can work within a common institutional language. And it maintains a performance record over time, so that work can be allocated according to demonstrated capability.

The project applies the equivalent three measures.

- **A single standard.** Every output, regardless of which assistant produced it, is evaluated against the same contract. The system does not adapt to the assistant; the assistant's output is required to satisfy the standard. This is the primary normalizing mechanism, and it is a direct consequence of the architecture described in Section 4 rather than an additional capability.
- **A translation layer.** A lightweight, assistant-specific adapter reconciles differences in interface and convention, presenting a uniform internal representation to the validation engine. The engine itself remains entirely unaware of which assistant produced the output it is evaluating.
- **An empirical performance record.** Using the historical record described in Section 6, the system accumulates factual observations regarding the performance of each assistant across different categories of task. These observations are statistical summaries derived from recorded outcomes, not learned models, and they

permit the system to adapt its behaviour over time without any loss of determinism in the validation decision itself.

The Essential Distinction

The system does not learn to predict what each assistant will produce. It holds every assistant to one explicit standard, translates between conventions where necessary, and maintains a factual record of observed performance. What appears externally as adaptation is arithmetic performed over recorded data, and the validation verdict itself remains entirely deterministic.

8. How the Three Domains Relate

The three problem domains are frequently presented as a list, which obscures an important structural property: they are not parallel features but layered capabilities, and the dependencies between them determine the order in which the system must be constructed.

Domain	Depends Upon	Contributes To	Nature of the Relationship
Core enforcement	Nothing — it is the foundation	All three domains	Provides the deterministic verdict upon which every other capability is constructed
Token optimization	Core enforcement	Overall system economics	The elimination of wasted corrective cycles is a direct consequence of deterministic detection
Tracking history	Core enforcement	Multi-agent normalization; economical context	The event record is the data foundation from which performance observations are derived
Multi-agent normalization	Core enforcement; tracking history	Consistency of outcome across the organization	Requires accumulated historical data before performance observations become meaningful

8.1 The Combined Effect

The relationship may be stated as a single proposition. The historical record produces factual observations regarding assistant performance; those observations inform the translation layer; the translation layer delivers normalized output to the deterministic validation engine; and the economic benefit produced by that engine therefore applies uniformly regardless of which assistant was used, because the same contract judged every one of them identically.

The practical consequence for project planning is that the construction order is not a matter of preference. The enforcement foundation must precede everything. The historical record must exist and must have accumulated data before performance observations can be meaningful. Any attempt to construct the normalization capability first would produce a component with no data to operate upon.

9. Conceptual Working of the System

The following description traces the operation of the system at a conceptual level. Technical detail is deliberately omitted here and is provided in the accompanying technical specification.

9.1 Preparation

The engineering team authors contracts expressing the organization's requirements and places them under version control alongside the source code. When the system starts, these contracts are loaded and held in memory so that subsequent evaluation does not require them to be retrieved again. Where the contracts are managed centrally, the system retains a local copy so that it continues to operate correctly if the central service becomes unavailable.

9.2 Evaluation

When an output requiring validation is produced, the system performs a sequence of steps. It first confirms that the output is within acceptable size limits, so that an unreasonably large input cannot consume disproportionate resources. It then retrieves the applicable contract. It evaluates the output against that contract using a fixed sequence of rules, executed within a bounded time allowance so that validation cannot delay the host application indefinitely. It records the outcome for later analysis. Finally, it applies the configured enforcement policy to determine whether the output proceeds, proceeds with a recorded warning, or is blocked.

9.3 Correction

Where an output does not conform, the result identifies the specific requirements that were violated. This information is returned to the assistant in a form suitable for direct consumption, enabling a targeted correction rather than an undirected further attempt. The corrected output is evaluated again. The cycle repeats until the output conforms or a configured limit is reached. Because each evaluation is deterministic, the cycle is predictable and cannot continue indefinitely without detection.

9.4 Accumulation

Every evaluation, whether successful or not, contributes a record to the historical log. Over time this log becomes the basis for recall of project context, identification of recurring problems, and the performance observations described in Section 7. The system therefore becomes more informative with use, while the cost of each individual evaluation remains constant.

9.5 Modes of Integration

Two distinct modes of integration are contemplated, reflecting the two ways in which AI systems are used in practice.

- **Direct integration.** Where the development team invokes an AI model directly from within its own application code, the validation engine can be positioned between the application and the model, evaluating each response as it is returned.
- **Tool-based integration.** Where the team uses a self-contained AI coding assistant that operates independently, the validation engine is instead exposed as a service that the assistant consults during its own operation. The assistant retains control of its process and requests a verdict before committing a change. This mode is the more significant of the two for the intended market, since it accommodates the assistants that are most widely used in practice.

10. Intended Users and Usage Scenarios

10.1 User Categories

User Category	Principal Concern	Benefit Obtained from the System
Individual developer	Producing acceptable work quickly without repeated correction	Immediate, specific feedback on non-conforming output; fewer corrective cycles
Technical lead / architect	Preserving architectural integrity as the codebase and team grow	Architectural rules enforced automatically and uniformly rather than through review alone
Engineering manager	Consistency of output across a team using varied tooling	A single standard applied to all work irrespective of the assistant used
Compliance / security function	Demonstrable evidence that policy is enforced	A deterministic, permanently recorded audit trail of every evaluation performed
New team member	Understanding the system and its history quickly	Access to accumulated project context and recorded decisions

10.2 Representative Scenarios

Scenario A — Preservation of Architectural Boundaries

An organization maintains a layered architecture in which certain categories of module are forbidden from depending on certain others. A developer instructs an AI assistant to add a feature. The assistant produces a change that introduces a forbidden dependency. Before the change is committed, the assistant consults the validation engine, which identifies the violation and reports the specific rule breached. The assistant revises the change accordingly. The forbidden dependency never enters the codebase, and no human review was required to prevent it.

Scenario B — Consistency of Structured Output

A service returns structured data consumed by several downstream systems. A contract specifies the required fields, their types and the permitted values of a constrained field. An AI-generated modification alters the response in a manner inconsistent with the contract. Validation detects the inconsistency at the point of generation rather than at the point of downstream failure.

Scenario C — Resumption of Work

A developer returns to a project after an absence, or inherits work from a colleague. Rather than reconstructing the state of the work by inspection, the developer and the AI assistant obtain a compact summary of recent activity, recurring issues and relevant recorded decisions from the historical record, and resume work directly.

11. Distinguishing Characteristics

Several properties differentiate the proposed system from adjacent categories of tooling. They are stated here at a conceptual level; their technical realization is described in the accompanying specification.

Characteristic	Description	Why It Matters
Determinism	Validation verdicts are produced by program logic, not by a probabilistic model	Enables policy, audit and trust; identical inputs always yield identical verdicts
Enforcement rather than observation	The system is capable of blocking non-conforming output, not merely reporting it	Monitoring tools record problems; an enforcement layer prevents them
Human-authored standards	Requirements are expressed in contracts written and controlled by engineers	Engineering authority remains with the engineering team
Architectural awareness	Rules may express structural relationships, not only data shape	Addresses architectural drift, which conventional data validation does not
Operation within the assistant's workflow	Validation occurs during generation rather than after it	Prevents non-conforming output rather than detecting it later
Assistant independence	The evaluation core is unaware of which assistant produced the output	Supports heterogeneous tooling without duplicated logic
Failure containment	The system degrades in a controlled manner and does not disrupt its host application	A governance tool that destabilizes production would not be adopted

12. Project Scope and Current Status

12.1 Development Status

The project is under active development. The core validation engine — the component responsible for the deterministic evaluation described in Section 4 — has been implemented and is supported by an extensive automated test suite covering both ordinary operation and deliberately adversarial conditions. The supporting infrastructure required for the engine to operate safely within a production application, including mechanisms for bounding execution time, tolerating the unavailability of external services, and safeguarding sensitive information, has likewise been implemented.

Work is presently focused on completing and hardening this foundation prior to the construction of the capabilities described in Sections 5 to 7, which build upon it. This sequencing reflects the dependency structure set out in Section 8.

12.2 Scope of the Present Academic Submission

For the purposes of the Software Project Management course, the project is treated as a defined engineering undertaking with documented objectives, stakeholders, deliverables, schedule and risks. The emphasis of the academic work is the application of project management practice — planning, estimation, documentation, risk management and progress tracking — to a genuine software product rather than to a hypothetical scenario.

The team considers this an advantage. A real system imposes real constraints: genuine technical dependencies that determine sequencing, genuine risks with genuine consequences, and genuine trade-offs between scope and schedule. These are the conditions in which project management practice is meaningfully exercised.

12.3 Boundaries of the Present Documentation

The documentation accompanying this submission describes the system at the level of concept, architecture and requirements. It intentionally does not include detailed implementation specifics, proprietary optimization techniques, competitive analysis or commercial strategy. This boundary is appropriate to the purpose of the submission, which is to demonstrate the application of project management practice to a defined engineering undertaking.

13. Conclusion

The adoption of AI coding assistants has produced a measurable increase in the rate at which software is produced, without a corresponding increase in the assurance that what is produced conforms to the standards of the organization consuming it. The existing controls — principally human review — were designed for a rate of production that no longer applies.

Congine proposes a specific and deliberately constrained response to this condition. Organizational requirements are expressed as contracts authored by engineers. Every relevant output is evaluated against the applicable contract by a rule-based engine that produces the same verdict for the same input on every occasion. Enforcement is graduated, so that the system may be introduced into an existing codebase without disruption. The evaluation is rapid enough to operate within the development loop and is designed to fail in a controlled manner rather than to destabilize the application in which it operates.

Three connected capabilities extend this foundation: reduction of the economic cost of AI interaction, achieved substantially as a consequence of eliminating wasted corrective cycles; preservation of project history and decision rationale in a durable record; and the normalization of work produced by different assistants through a single shared standard rather than through model training. These capabilities are layered rather than parallel, and their dependencies determine the order of construction.

The property that unifies the design is determinism. It is the reason the system can support organizational policy, the reason its verdicts can be audited, and the reason its economic benefit is reliable rather than incidental. Every significant design decision in the project is traceable to the preservation of that property.

Summary

Congine is a deterministic governance layer for AI-assisted software development. It enforces human-authored standards on machine-generated output, automatically and consistently, and in doing so reduces the cost of AI interaction, preserves project knowledge that would otherwise be lost, and produces consistent results across a team using varied tooling.

14. Glossary of Terms

Term	Definition
AI coding assistant	A software tool employing an artificial intelligence model to generate or modify source code and related artifacts on a developer's behalf.
Architectural drift	The gradual divergence of a codebase from its intended design, arising from an accumulation of individually minor deviations.
Contract	A machine-readable document, authored by engineers, specifying the requirements an output must satisfy in order to be accepted.
Determinism	The property whereby a given input evaluated against a given ruleset always produces an identical result.
Enforcement policy	The configured response applied when an output is found not to conform: permissive, strict or observational.
Governance gap	The disparity between the rate at which AI systems generate output and the rate at which that output can be reviewed and assured.
Historical record	The durable, chronologically ordered log of validation events maintained by the system.
Normalization	The process of causing outputs from different AI assistants to satisfy one common standard.
Output firewall	The governing analogy for the system: a control positioned between a producer of output and its consumer, admitting only conforming output.
Token	The unit in which text is measured for the purpose of charging for interaction with an AI language model.
Validation engine	The component responsible for evaluating an output against a contract and producing a structured verdict.

End of Document — CGN-DOC-01, Project Concept Brief, Version 1.0